Diseño de Algoritmos – Práctica 1

Algoritmos devoradores

Francisco Palomo Lozano

Version 2.5

1. Se trata de simular la operación de reintegro en un cajero automático. Para ello, se requiere que programe el algoritmo devorador de cambio de moneda empleando internamente un montículo para mejorar su eficiencia.

   El cajero automático se carga inicialmente con fajos de billetes de cada valor, cada uno con un cierto número de unidades. La clase y el tipo que se definen a continuación sirven para representar estos conceptos:

   ```
   class fajo {
   public:
     fajo(): v{0}, u{0} {}
     fajo(unsigned int v, unsigned int u): v{v}, u{u} {}
     unsigned int valor() const { return v; }
     unsigned int& valor() { return v; }
     unsigned int unidades() const { return u; }
     unsigned int& unidades() { return u; }
   private:
     unsigned int v, u;    // Valor y número de unidades de cada billete.
   };

   using fajos = std::vector<fajo>;
   ```

   La función que programa el algoritmo deberá tener el siguiente prototipo:

   ```
   fajos reintegrar(const fajos& caja, unsigned int cantidad);
   ```

   Esta función recibe los fajos que están en la caja y la cantidad a reintegrar, y devuelve los fajos que componen el reintegro. Si no se puede reintegrar la cantidad deseada no devuelve ninguno, en cuyo caso, el resultado debe corresponder a un contenedor vacío.

Obsérvese que la función no debe modificar la caja inicial: en una aplicación real la modificación se realiza externamente a la función, tras garantizarse la entrega física del reintegro al cliente.

El programa principal solicitará la cantidad a reintegrar y mostrará la composición del reintegro o indicará, en su caso, que no puede proporcionarlo. La información relativa a la caja inicial se leerá de un fichero, `caja.dat`, en el que cada línea contendrá la información relativa a un fajo (el valor del billete y su número de unidades).

Por ejemplo:

```
10    21
20    11
50    16
100   10
```

La entrada de datos puede realizarse a través de un flujo de entrada asociado al fichero con un objeto de la clase `ifstream`, declarada en `<fstream>`. Suponiendo el uso del espacio de nombres `std`, las siguientes líneas ilustran este proceso:

```
ifstream datos("caja.dat");
fajos caja;
datos >> caja;
```

Para que la última línea tenga sentido es necesario sobrecargar el operador `>>` para trabajar con fajos:

```
istream& operator >>(istream& fe, fajo& f)
{
  return fe >> f.valor() >> f.unidades();
}

istream& operator >>(istream& fe, fajos& f)
{
  istream_iterator<fajo> ife(fe), fin;
  copy(ife, fin, back_inserter(f));
  return fe;
}
```

Por último, es conveniente sobrecargar el operador `<<` para poder presentar los resultados cómodamente:

```
ostream& operator <<(ostream& fs, const fajo& f)
{
  return fs << f.unidades() << " billete"
            << (f.unidades() == 1 ? "" : "s") << " de " << f.valor();
}
```

```
ostream& operator <<(ostream& fs, const fajos& f)
{
  copy(begin(f), end(f), ostream_iterator<fajo>(fs, "\n"));
  return fs;
}
```

Estas sobrecargas de los operadores >> y << requieren previamente la inclusión de las cabeceras estándar <iostream>, <iterator> y <algorithm>.

2. Defina la clase que se declara a continuación:

```
class Particion {
public:
  Particion(int n);
  int buscar(int i) const;
  void unir(int i, int j);
private:
  std::vector<int> p;
};
```

Esta clase ha de permitirnos trabajar con estructuras de partición, empleando como representación interna un bosque con control de alturas en el que las alturas se representarán implícitamente.

El constructor de la clase creará la partición inicial de n elementos. Se supondrá que $n > 0$ y que los elementos se numeran de 0 a $n - 1$. Por lo tanto, en `buscar` y `unir` puede suponerse que $0 \leq i, j < n$. Por último, en `unir` puede suponerse que $i \neq j$ y que $p[i], p[j] < 0$, es decir, que i y j son raíces distintas en el bosque representado por p.

3. El objetivo de este ejercicio es programar el algoritmo de Kruskal empleando la técnica de preordenación. En los listados adjuntos se definen cuatro clases paramétricas (`Arista`, `Grafo`, `AristaPonderada` y `GrafoPonderado`) que permiten representar grafos de manera muy apropiada para la programación del algoritmo.

Programe el algoritmo empleando las estructuras de partición del ejercicio anterior. Siga el esqueleto que se presenta a continuación, donde se supone que `G` es un `GrafoPonderado` abstracto.

```
template <typename G>
G Kruskal(const G& g)
{
  using std::size_t;
  using std::list;
  using Arista = typename G::Arista;
  size_t n = g.vertices();              // Número de vértices.
```

```
  G s(n);                                  // Grafo solución.
  Particion p(n);                          // Partición inicial.
  list<Arista> c(g.aristas());             // Lista de candidatos.

  // ...

  return s;
}
```

Finalmente, cree un programa de prueba que calcule el árbol de expansión mínimo de un grafo almacenado en un fichero de nombre `grafo.dat`. La primera línea del fichero indicará el número de vértices. Tras esta, cada línea representará una arista ponderada (vértices y peso). Por ejemplo:

```
7
0 1 1.0
0 3 4.0
1 2 2.0
1 3 6.0
1 4 4.0
2 4 5.0
2 5 6.0
3 4 3.0
3 6 4.0
4 5 8.0
4 6 7.0
5 6 3.0
```

Las siguientes sobrecargas de los operadores >> y << le resultarán muy útiles para realizar operaciones de entrada y salida con grafos representados en este formato.

```
template <typename V, typename P>
istream& operator >>(istream& fe, AristaPonderada<V, P>& a)
{
  return fe >> a.primero() >> a.segundo() >> a.peso();
}

template <typename V, typename A>
istream& operator >>(istream& fe, Grafo<V, A>& g)
{
  size_t n = 0;
  fe >> n;
  g = Grafo<V, A>(n);
  V u{}, v{};
  A a(u, v);
  while (fe >> a)
    g.insertar(a);
```

```
  return fe;
}

template <typename V, typename P>
ostream& operator <<(ostream& fs, const AristaPonderada<V, P>& a)
{
  return fs << a.primero() << ' ' << a.segundo() << ' ' << a.peso();
}

template <typename V, typename A>
ostream& operator <<(ostream& fs, const Grafo<V, A>& g)
{
  fs << g.vertices() << '\n';
  auto& a = g.aristas();
  copy(begin(a), end(a), ostream_iterator<A>(fs, "\n"));
  return fs << flush;
}
```

4. Modifique el programa anterior para que emplee montículos en lugar de preordenación y discuta sus posibles ventajas.

5. Analice experimentalmente el tiempo en el peor caso de las dos versiones del algoritmo de Kruskal programadas en los ejercicios previos. Preste especial atención al diseño del experimento, que deberá describir en detalle. Finalmente, verifique que los resultados obtenidos están de acuerdo con los que se obtienen mediante el análisis teórico.